Spatial Filtering & Convolution

Week 3 — Session 2: Sharpening, Nonlinear & Advanced Filtering

Practical Image Processing

Lecture 3.2 ▪ 60–70 min

Recap of Session 1

- **Convolution**: slide a kernel, multiply the neighbourhood, sum, write the result.
- The **kernel is the algorithm**; a blur kernel sums to 1, an edge kernel sums to 0.
- We smoothed with **box** and **Gaussian**, and saw the detail-vs-noise trade-off.

Today (Session 2)

The other direction — *sharpening* — plus nonlinear filters, the frequency view, speed via separability, and the CNN bridge.

Roadmap — Session 2

Sharpening

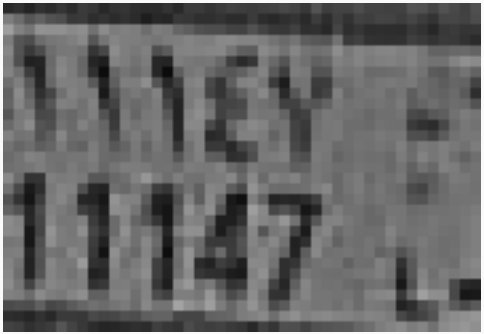
Nonlinear filters

Deep dive

Wrap-up

Sharpening

Sharpening = add back the detail



original



- **Unsharp mask**:
  sharp $=$ img $+ \lambda($img $-$ blur$)$.
- The (img $-$ blur) term *is* the high-frequency detail.
- Adds it back to emphasize edges — the digits' strokes stand out.

Why “unsharp” means sharpen

The detail signal

$\text{detail} = \text{img} - \text{blur}(\text{img}) \qquad \text{sharp} = \text{img} + \lambda \cdot \text{detail}$

- Subtracting a blurred (“unsharp”) copy leaves only what the blur removed — the fine detail.
- Add a scaled copy of that detail back and edges get exaggerated.
- λ is the strength: $\lambda = 1$ is moderate; $\lambda = 3$ is aggressive.

Key idea

The name is historical (a darkroom trick); the math is just image + its own high-pass.

The sharpen kernel is unsharp mask in one step

Combine identity and (identity − box) into a single kernel:

$$\underbrace{\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}}_{\text{identity}} + \lambda\Big(\text{id} - \text{blur}\Big) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- Centre 5 = the pixel plus its own boosted detail.
- Sides -1 = subtract the neighbours (the local average).
- Sums to $1 \Rightarrow$ flat regions unchanged, only edges pop.

Note

A single convolution does the whole unsharp mask when the blur is a small box — no separate subtract step.

Warm-up exercise (8 min)

Sharpen by hand

A patch with an edge (dark left, bright right) and the sharpen kernel:

$$P = \begin{bmatrix} 10 & 10 & 200 \\ 10 & 10 & 200 \\ 10 & 10 & 200 \end{bmatrix} \qquad K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

1. Compute the output at the centre pixel (value 10, its right neighbour is 200).
2. Is the result pulled toward the dark side or the bright side? What does that do at an edge?

Debrief

- Centre: $5(10) - 10 - 10 - 10 - 200 = 50 - 230 = -\mathbf{180}$, clamped to $\mathbf{0}$.
- The dark side of the edge is driven *darker* (to 0); by symmetry the bright side is driven brighter.
- That widening gap is exactly the bright/dark rim you see along a sharpened edge — enhanced local contrast.

Key idea

Sharpening pushes the two sides of an edge apart. Push too hard and the rim becomes a visible halo.

High-boost & the Laplacian

High-boost generalizes the centre weight:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & A{+}4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$A = 1$ is the plain sharpen; larger A keeps more of the original.

The **Laplacian** is the pure high-pass ($\sum = 0$):

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Sharpen = image + Laplacian. It measures how much a pixel differs from its surround.

Note

Same family, one knob: sharpen, high-boost and Laplacian differ only in the centre weight.

Sharpening's catch

- It amplifies *everything* high-frequency — including noise and JPEG blocks.
- Over-sharpening creates **halos** (bright/dark rings) around edges.
- Denoise first, then sharpen gently. Never sharpen a noisy image blindly.

✕ Pitfall

Sharpening a raw, noisy meter crop turns sensor speckle into a field of bright dots that wrecks the threshold. Smooth (Session 1) *then* sharpen.

Key idea

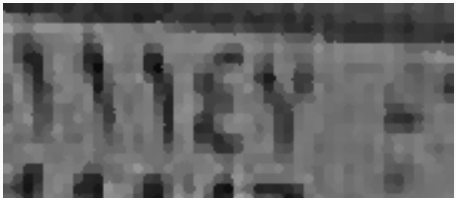
Blur and sharpen are opposites, but they're both convolution — just different kernels.

Nonlinear filters

Nonlinear filter: the median



salt-and-pepper noise



- A **median** filter takes the median of the neighbourhood — not a weighted sum.
- It kills salt-and-pepper noise while keeping edges sharp (a Gaussian would smear them).
- Nonlinear $\rightarrow$ not convolution, but same "slide a window" idea.

Why the median beats the mean on spikes

Neighbourhood values (one "salt" spike):

$$\{12,\ 14,\ 13,\ \mathbf{255},\ 15,\ 13,\ 14,\ 12,\ 15\}$$

- **Mean** $= 40.3$ — the spike dragged it up.
- **Median** (sorted, middle) $= 14$ — the spike is simply *discarded*.

✔ Good practice

The median is an **order statistic**: one wild outlier cannot move it, because it never enters the arithmetic — it just sits at the end of the sorted list.

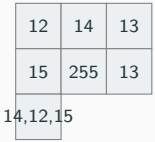
Key idea

Mean averages the outlier in; median throws it out. That is why median wins on impulsive noise.

The rank-filter family

Sort the neighbourhood, then pick a position:

- **min** filter (erosion-like): darkens, removes bright specks.
- **max** filter (dilation-like): brightens, removes dark specks.
- **median**: the middle — balanced impulse removal.



Note

Min/max are the seed of **morphology** (Week 4). Median sits between them and is the workhorse for salt-and-pepper.

Bilateral filter: smooth but keep edges

- Gaussian blur smears edges; the **bilateral** filter weights by *both* distance *and* intensity similarity.
- Result: flat regions get smoothed, edges stay crisp — great before thresholding a document.
- Slower, but often the right smoother for DocScan: `cv2.bilateralFilter`.

Math

$$w(i,j) = \underbrace{e^{-\frac{i^2+j^2}{2\sigma_s^2}}}_{\text{spatial}} \underbrace{e^{-\frac{(f_p - f_q)^2}{2\sigma_r^2}}}_{\text{range}}$$

Good practice

When you need to denoise *without* blurring the digit edges, reach for bilateral (or median).

Tuning the bilateral filter

- σ_s (spatial): how far a neighbour can be and still count — like a Gaussian radius.
- σ_r (range): how different in intensity a neighbour can be and still count.
- Large $\sigma_r \rightarrow$ behaves like a plain Gaussian (edges start to smear again).
- Small $\sigma_r \rightarrow$ only near-equal pixels mix, so edges are strongly preserved.

✗ Pitfall

It is expensive: cost grows with window size and it is not separable. Don't run it on a full frame every loop iteration — crop to the region of interest first.

Key idea

σ_r is the "edge sensitivity" knob — the whole reason to prefer bilateral over Gaussian.

A zoo of kernels & filters

Filter	Effect	Linear?
all $1/n$	box blur (average)	yes
Gaussian weights	smooth blur	yes
centre 5, sides -1	sharpen (high-pass)	yes
$[-1, 0, 1]$ rows	horizontal gradient (edges, Week 5)	yes
identity + (id $-$ blur)	unsharp mask	yes
median of window	impulse-noise removal	no
bilateral	edge-preserving smooth	no

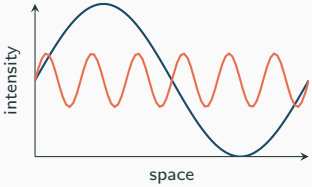
Key idea

You don't memorize kernels — you understand what "average" vs "difference" vs "order statistic" does.

Deep dive

Frequency view: low-pass vs high-pass

- Smoothing keeps **low frequencies** (slow changes), removes high (a **low-pass** filter).
- Sharpening/edges keep **high frequencies** (fast changes) — a **high-pass** filter.
- Week 6 makes this literal with the Fourier transform; today it's intuition.



blue = low freq (kept by blur)

coral = high freq (kept by sharpen)

✔ Good practice

"Blur = low-pass, edges = high-pass" is a lens you'll use for the rest of the course.

Separability & speed

- A separable 2D kernel = outer product of two 1D kernels; apply them in sequence.
- A $k \times k$ blur costs k^2 multiplies per pixel; separated, only $2k$.
- OpenCV does this automatically for Gaussian; know it so you write fast filters.

Cost, $k = 9$

naive: $k^2 = 81$/pixel

separated: $2k = 18$/pixel

$\Rightarrow 4.5\times$ faster, and it grows with k.

Note

The 3×3 Gaussian $\frac{1}{16}[1, 2, 1; 2, 4, 2; 1, 2, 1]$ is $\frac{1}{4}[1, 2, 1]$ applied horizontally then vertically — proof by construction.

Convolution is the bridge to deep learning

- A CNN layer (Week 13) is exactly this: convolve the image with small kernels.
- The difference: the network *learns* the kernel values from data instead of you designing them.
- Everything you learn about kernels this week is what a CNN's first layers do.

Key idea

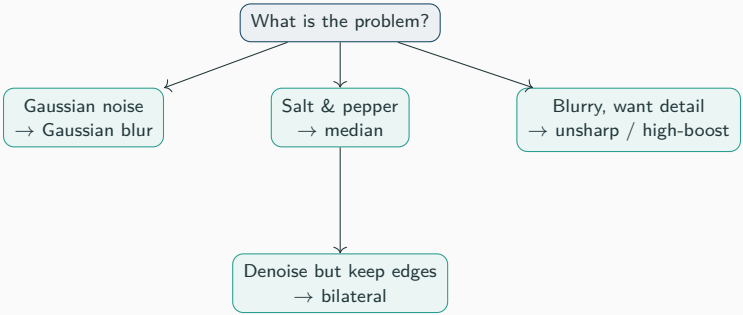
Classical filters are hand-designed kernels; CNNs are learned kernels. Same operation.

Filtering in code — the full toolbox

```
box    = cv2.blur(img, (5, 5))
gauss  = cv2.GaussianBlur(img, (7, 7), 1.5)
median = cv2.medianBlur(img, 3)              # nonlinear, impulse noise
bilat  = cv2.bilateralFilter(img, 9, 75, 75)# edge-preserving smooth
k = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharp  = cv2.filter2D(img, -1, k)            # any custom kernel
```

`cv2.filter2D` convolves with any kernel you give it — the general tool the others are shortcuts for.

Which filter, when? — a decision guide



Key idea

Match the filter to the *noise/goal*, not to habit. The zoo exists because no one filter wins everywhere.

Common pitfalls, collected

✕ Sharpen-then-notice-the-noise

Sharpening amplifies noise. Always denoise *first*, then sharpen gently.

✕ Median for the wrong noise

Median is superb for impulse noise but poor for fine Gaussian grain — use Gaussian/bilateral there.

✕ Over-blur before threshold

Enough blur to kill noise, not so much the digit strokes merge — tune σ on your own images.

Linear vs nonlinear: what you keep or lose

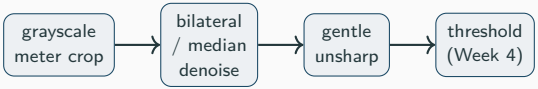
	Linear (convolution)	Nonlinear (median/bilateral)
Separable / very fast	often yes	no
Superposition holds	yes	no
Edge preservation	poor (blur)	good
Impulse-noise removal	poor	excellent
Analyzable in frequency	yes (Week 6)	not directly

Key idea

Linear filters are fast and analyzable; nonlinear ones protect edges. Pick the property you need.

Wrap-up

Putting it together: the DocScan pre-filter chain



- Denoise with an *edge-preserving* filter so the digit strokes survive.
- A light sharpen recovers stroke contrast the denoise softened.
- Only then threshold — a clean, sharp mask is far easier to read.

Key idea

Filters are rarely used alone; they form a short pipeline tuned to your images.

What you should now be able to do

- Sharpen with the unsharp mask, derive its kernel, and predict the halos it causes.
- Choose between mean, median and bilateral for a given kind of noise.
- See the low-pass/high-pass view and exploit separability for speed.
- Connect classical kernels to the learned kernels of a CNN.

This week's deliverable

Due before Week 4 (see `project/`)

On real meter/plate images:

1. Implement a 3×3 convolution *from scratch* (NumPy, Session 1) and match `cv2.filter2D`.
2. Compare box vs Gaussian blur at several sizes; discuss detail loss vs noise removal.
3. Sharpen a plate with unsharp mask; find where it starts creating halos.
4. Add salt-and-pepper noise and compare median vs Gaussian denoising.
5. Note: what blur helps your DocScan threshold, and what over-blurs the digits?

The kernel is the algorithm.

Blur, sharpen, edges — all the same operation with different numbers.